Documentación de GrowlToast Notification ~ Versión 1.0

Librería: `ifclsgrowltoast.vcx` · Lenguaje: Visual FoxPro 9.0 · Contenido: guía de uso y referencia completa de todas las clases

1. Visión General

GrowlToast es una librería de notificaciones emergentes (toast) al estilo Growl para Visual FoxPro 9. Los toasts aparecen apilados en la esquina superior derecha del formulario, entran con una animación deslizante, muestran una barra de cuenta regresiva en la parte inferior y luego se desvanecen (fade) cuando se agota el tiempo, al hacer clic o al cerrarlos.

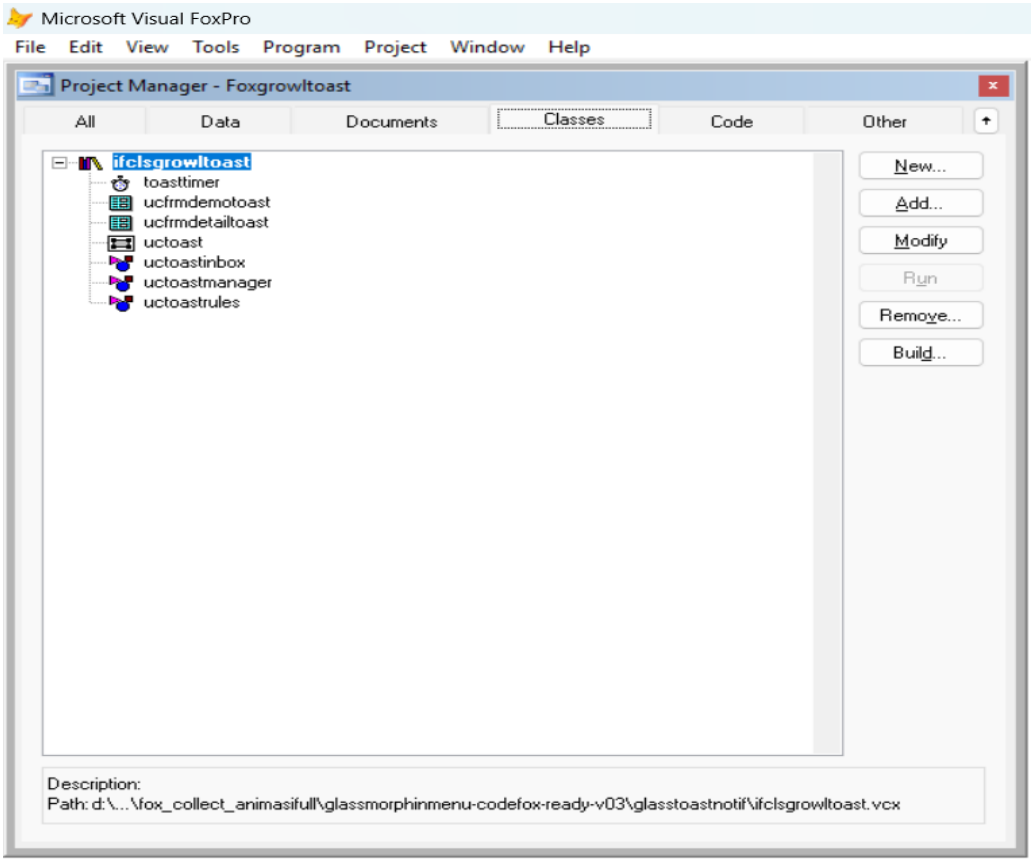
Características principales

- Los toasts se dibujan como objetos hijos dentro del formulario anfitrión (host), no como ventanas independientes. Por eso no roban el foco ni requieren APIs de ventana adicionales.
- Cuatro tipos de color: 1 verde (éxito/general), 2 azul (información), 3 naranja (advertencia), 4 rojo (peligro).
- Cola automática: si hay demasiados toasts, el resto espera y aparece por turnos (stagger).
- El hover (mouse sobre el toast) pausa la cuenta regresiva. Clic en el toast = abrir detalle. Botón X = cerrar.
- Los toasts que expiran sin respuesta vuelven a aparecer tras un intervalo determinado (requeue). Ideal para alertas que aún no han sido atendidas.
- Tres fuentes de notificación: (1) llamada directa a `Notify()`, (2) bandeja de entrada en tabla DBF que otros módulos pueden llenar, (3) reglas automáticas de stock y cuentas por cobrar.

Lista de clases

Clase	Deriva de	Función
`ToastTimer`	Timer	Timer de 16 ms del manager, llama a `Tick()`
`ucToast`	Container	Aspecto de una tarjeta toast (ícono, título, mensaje, barra de tiempo)
`ucToastManager`	Custom	El cerebro de la librería: cola, animación, posición, eventos
`ucToastInbox`	Custom	Bandeja de notificaciones basada en tabla DBF

Clase	Deriva de	Función
`ucToastRules`	Custom	Reglas automáticas: stock agotado/mínimo y cuentas por cobrar vencidas
`ucFrmDemoToast`	Form	Formulario de demostración y ejemplo de host
`ucFrmDetailToast`	Form	Formulario de detalle de stock crítico y cuentas por cobrar vencidas



2. Arquitectura y Flujo de Trabajo

```
Otros módulos / Bandeja DBF / Reglas
        |  Notify(...)
        v
ucToastManager.oQueue --(Fill: hay espacio libre y ya pasó
el tiempo de espera)--> oActive
```

```
|          Spawn: host.AddObject('oToast...', 'ucToast')

|
ToastTimer (16 ms) -> Tick() -> ucToast.Step() : animación,
hover, cuenta regresiva
        |
clic / cerrar -> PostEvent -> oEvents --(timer del host 100
ms: DispatchEvents)--> host.OnToastClick / host.OnToastClose
```

Puntos importantes del flujo:

1. `Notify()` solo agrega el elemento a la cola (`oQueue`). No dibuja nada de inmediato.
2. `ToastTimer` late cada 16 ms (unos 60 fps) y llama a `Tick()` del manager. Este timer solo está activo mientras haya toasts activos o en cola, y se apaga solo cuando todo queda vacío.
3. `Tick()` mueve todos los toasts activos, pasa los elementos de la cola a la pantalla (`Fill`/`Spawn`) y luego reacomoda las posiciones (`Relayout`).
4. El clic y el cierre no llaman directamente al host. Se registran como eventos (`PostEvent`) y luego los envía `DispatchEvents()`, que es invocado por el timer del propio host. Así se evita que el código del host se ejecute en medio del proceso de animación.
5. `Watchdog()` rescata el sistema si el timer se bloquea (por ejemplo, por un error): si pasa más de 1 segundo sin latido, el timer se reactiva y se fuerza la ejecución de `Tick()`.

3. Preparación

- **Cargar la librería**: `SET CLASSLIB TO ifclsgrowltoast.vcx ADDITIVE`
- **Tablas**: `toast_inbox.dbf`, `toast_barang.dbf`, `toast_piutang.dbf` se crean automáticamente si no existen (solo se necesitan si se usan Inbox y Rules).
- **Fuente**: Webdings (incluida en Windows) se usa para los íconos y el botón X.
- **API de Windows**: `GetCursorPos` y `ScreenToClient` (user32) se declaran automáticamente en el `Init` del manager.
- **Requisitos del formulario host**: debe tener `HWnd`, `Width`, `Height`, `BackColor`, `LockScreen`, `AddObject`, `RemoveObject` (un formulario VFP normal ya los cumple). Los métodos `OnToastClick` y `OnToastClose` son opcionales, pero sin ellos el clic y el cierre no tienen efecto del lado del host.

4. Inicio Rápido (uso mínimo)

Agregue la propiedad `oToast = .NULL.` al formulario host, un timer `tmrEvt` (Interval = 100) y luego complete el siguiente código.

```
* Form.Load ----*
SET CLASSLIB TO ifclsgrowltoast.vcx ADDITIVE

* ---- Form.Init ----
This.oToast = CREATEOBJECT('ucToastManager', This)
This.oToast.nMaxVisible = 5        && máximo de toasts visibles a la
vez
This.oToast.nDefaultMs  = 10000    && duración predeterminada (ms)
This.oToast.Notify('Hola', 'Este es mi primer toast.', 1)

* ---- tmrEvt.Timer (Interval = 100) ----
IF NOT ISNULL(ThisForm.oToast)
    ThisForm.oToast.DispatchEvents()
    ThisForm.oToast.Watchdog()
ENDIF

* ---- Método del formulario: se llama al hacer clic en el toast ----
PROCEDURE OnToastClick
    LPARAMETERS tcKey, tcTitle, tcMsg
    =MESSAGEBOX('Toast con clic: ' + tcTitle)
ENDPROC

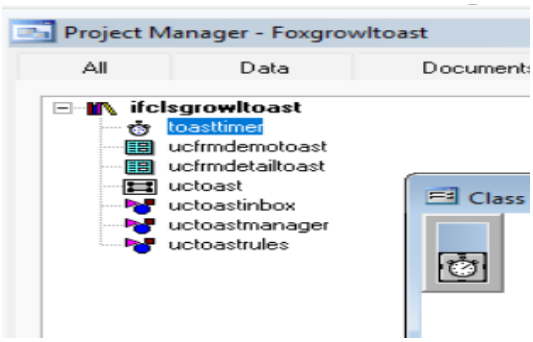
* ---- Método del formulario: se llama al presionar el botón X ----
PROCEDURE OnToastClose
    LPARAMETERS tcKey, tcTitle, tcMsg
ENDPROC

* ---- Form.Destroy (OBLIGATORIO) ----
IF NOT ISNULL(This.oToast)
    This.oToast.Shutdown()
    This.oToast = .NULL.
ENDIF
```

5. Referencia de Clases

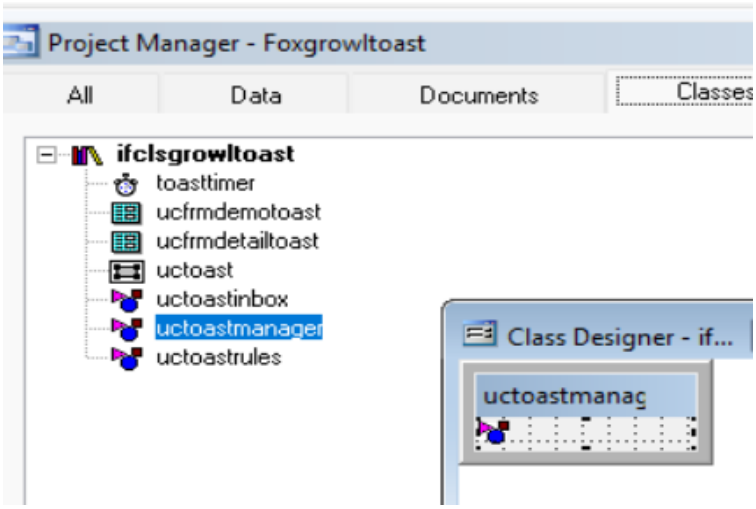
5.1 ToastTimer

Un pequeño timer (`Enabled = .F.`, 23x23) que el manager agrega como objeto llamado `tmr`. El evento Timer contiene una sola línea: `This.Parent.Tick()`. No lo usa directamente el programador.



5.2 ucToastManager

Creación : `CREATEOBJECT('ucToastManager', toHost)`, donde `toHost` = el formulario donde se mostrarán los toasts.



Propiedades configurables

Propiedad	Valor predeterminado	Descripción
`nMaxVisible`	5	Máximo de toasts visibles a la vez. También limitado por la altura del host (ver `GetCapacity`).
`nDefaultMs`	10000	Duración en pantalla cuando `Notify` no indica `tnMs`.
`nRequeueMs`	180000	Intervalo (ms) antes de volver a mostrar un toast expirado. El formulario demo lo sobrescribe desde el spinner.
`nMaxRepeat`	0	Límite de repeticiones del toast. 0 = sin límite.
`nGap`	10	Separación vertical entre toasts (px).
`nMargin`	16	Distancia desde el borde superior y derecho del host (px).
`nMoveMs`	450	Duración de la animación de desplazamiento vertical (ms).
`nFadeMs`	450	Duración de la animación de desvanecimiento

Propiedad	Valor predeterminado	Descripción
		(ms).
`nStaggerMs`	450	Intervalo mínimo entre la aparición de toasts nuevos.
`nStaggerOutMs`	500	Intervalo mínimo entre toasts expirados que comienzan a desvanecerse.
`cUserGlyph`	CHR(255)	Carácter Webdings predeterminado para el ícono (se muestra como un pájaro).
`nToastW`, `nToastH`	340, 84	Tamaño del toast. **Debe coincidir** con Width/Height de la clase `ucToast`.

Propiedades internas (no modificar desde fuera) : `oHost`, `oQueue`, `oActive`, `oEvents`, `nClock`, `nLastTick`, `nLastSpawn`, `nLastFadeStart`, `nSeq`, `lBusy`, `nBusySince`, `lDispatching`.

Métodos para el programador

Método	Función
`Notify(tcTitle, tcMsg, tnType, tcImage, tnMs, tcGlyph, tcKey, tcTime)`	Agrega una notificación a la cola. Devuelve la clave (key).
`Exists(tcKey)`	`.T.` si la clave existe entre los toasts activos o en la cola.
`UpdateText(tcKey, tcTitle, tcMsg)`	Cambia el título y el mensaje de un toast visible o en cola sin generar un toast nuevo.
`Dismiss(tcKey)`	Elimina de la cola y desvanece el toast activo con esa clave.
`ClearAll()`	Vacía la cola y desvanece todos los toasts activos.
`DispatchEvents()`	Envía al host los eventos de clic/cierre. Llamar desde el timer del host.
`Watchdog()`	Vigilante del timer. Llamar desde el timer del host junto con `DispatchEvents`.

Método	Función
`DiagText()`	Texto de diagnóstico: estado del timer, próxima aparición, intervalo de repetición.
`Shutdown()`	Apaga el timer, libera todos los toasts y la cola. Obligatorio en el `Destroy` del host.

Parámetros de `Notify`

Parámetro	Descripción
`tcTitle`	Título (en negrita).
`tcMsg`	Texto del mensaje (hasta 2 líneas, con ajuste automático).
`tnType`	1 verde, 2 azul, 3 naranja, 4 rojo. Predeterminado 1.
`tcImage`	Ruta de la imagen de avatar. Si el archivo existe, la imagen reemplaza al ícono glyph.
`tnMs`	Duración en pantalla (ms). Vacío o 0 = `nDefaultMs`.
`tcGlyph`	Un carácter Webdings para el ícono. Vacío = `cUserGlyph`.
`tcKey`	Clave única. Si la clave ya existe, la llamada se ignora (anti duplicados). Vacío = se genera automáticamente `I1`, `I2`, etc.
`tcTime`	Texto de la hora. Vacío = hora actual. Se muestra como `Pukul hh:mm` (literalmente, «a las hh:mm» en indonesio, según el código original).

Callbacks hacia el host (métodos del formulario host, con los mismos parámetros)

- `OnToastClick(tcKey, tcTitle, tcMsg)`: se hizo clic en el toast.
- `OnToastClose(tcKey, tcTitle, tcMsg)`: se presionó el botón X.

Los callbacks solo se invocan si el método existe en el host (se verifica con `PEMSTATUS`).

Métodos internos: `StartTimer`, `Tick`, `PostEvent`, `ReleaseFade`, `GetCapacity`, `Fill`, `Spawn`, `Relayout`, `Retire`, `RaiseEvent`, `Init`, `Destroy`. En resumen:

- `Tick`: un latido de animación. Calcula la diferencia de tiempo real, lee la posición del mouse (para el hover), llama a `Step` en cada toast y luego a

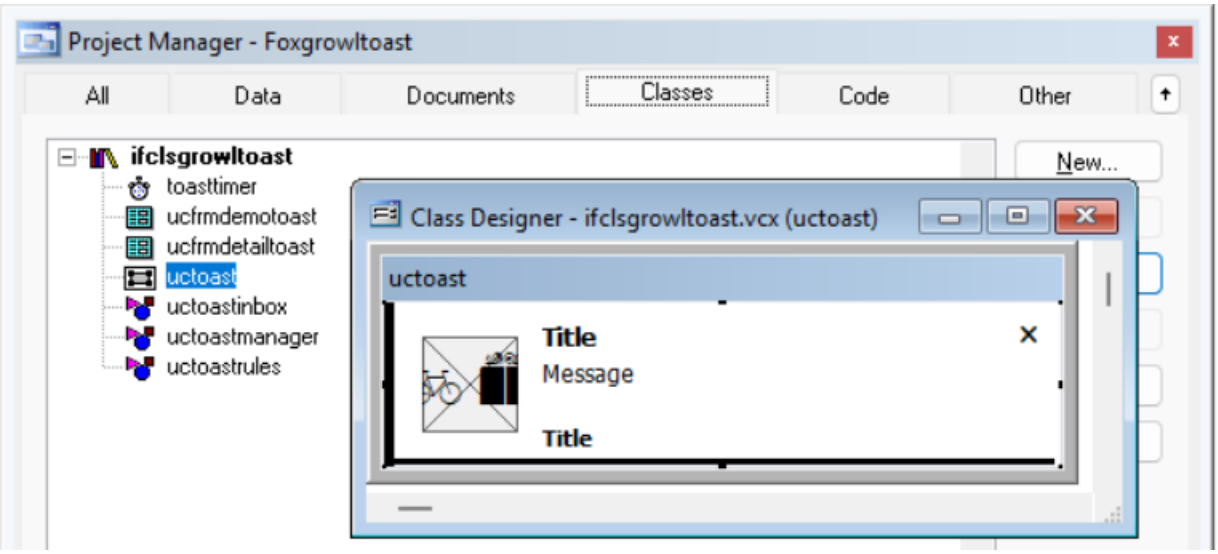
`ReleaseFade`, `Fill`, `Relayout`. Usa `LockScreen` para evitar parpadeos y el guardián `lBusy` para evitar reentradas.

- `GetCapacity`: cantidad de toasts que caben = el menor valor entre `nMaxVisible` y el resultado de dividir la altura del host.
- `Fill`: si aún hay espacio y pasó el intervalo stagger, toma un elemento de la cola cuyo `nWaitUntil` ya llegó y ejecuta `Spawn`.
- `Spawn`: crea un nuevo objeto `ucToast` en el host (`AddObject`), llama a `Setup` y lo ubica bajo el borde inferior del host (luego sube mediante animación).
- `Retire`: elimina el objeto toast del host. Si el toast expiró (no por clic ni cierre) y no ha superado `nMaxRepeat`, el elemento vuelve a la cola con `nWaitUntil = nClock + nRequeueMs`.
- `ReleaseFade`: los toasts expirados se liberan para desvanecerse de uno en uno, no todos a la vez, y no mientras tengan el mouse encima.

Datos de cada elemento (objeto Empty en `oQueue`/`oActive`): `cId`, `cKey`, `cTitle`, `cMsg`, `nType`, `cImage`, `cGlyph`, `nMs`, `cTime`, `nRepeat`, `nWaitUntil`, `cViewName`, `oView`.

5.3 ucToast

Contenedor de 340x84 que representa una tarjeta. Lo crea automáticamente `Spawn`; no hay que crearlo manualmente. Contiene: `shpAccent` (línea de color a la izquierda), `shpAvatar` + `lblGlyph` (ícono redondo), `imgUser` (imagen de avatar), `lblTitle`, `lblMsg`, `lblTime`, `lblClose` (botón X, Webdings), `shpBar` (barra de cuenta regresiva).



Color por tipo

`tnType`	Color de acento	Uso común
1 (u otro)	Verde RGB(46,204,113)	Mensaje general / éxito
2	Azul RGB(52,152,219)	Información
3	Naranja RGB(243,156,18)	Advertencia
4	Rojo RGB(231,76,60)	Peligro / crítico

Estado del toast (`nState`)

Valor	Significado
1	Visible con normalidad, la barra se reduce. Deja de contar mientras hay hover.
4	Tiempo agotado, esperando su turno para desvanecerse (o esperando a que el mouse se retire).
2	Desvaneciéndose mientras se desplaza 30 px a la derecha.
3	Terminado, listo para que el manager lo elimine.

Métodos

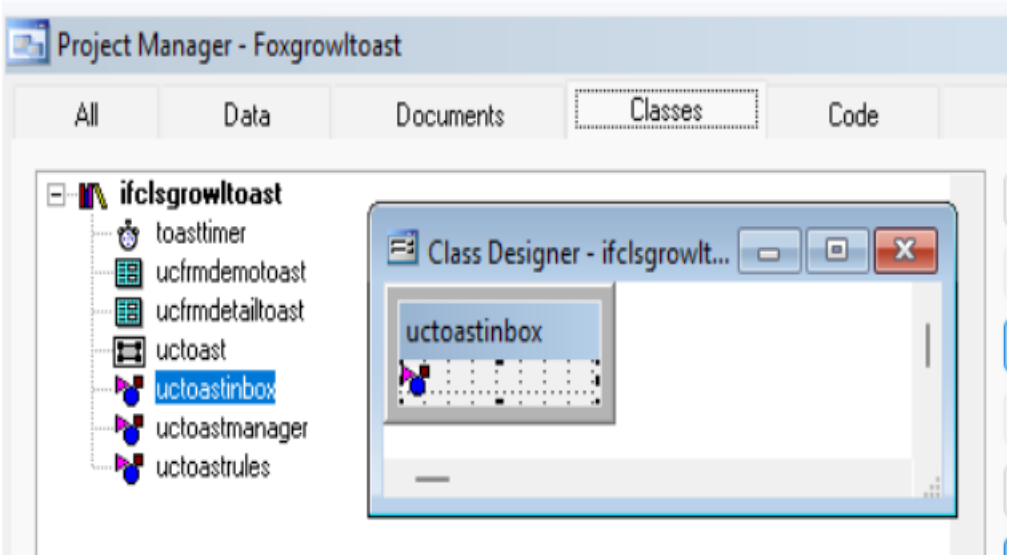
Método	Función
`Setup(...)`	Establece texto, colores, imagen/ícono y duración. Lo llama `Spawn`.
`ApplyColors(tnAlpha)`	Aplica los colores a todos los elementos según el nivel de visibilidad (1 = completo, 0 = invisible).
`Blend(tnA, tnB, tnT)`	Mezcla dos colores RGB con la proporción `tnT`.
`MoveTo(tnTo)`	Inicia la animación de desplazamiento hacia una nueva posición Top (easing cubic-out).
`SetHover(tlOn)`	Resalta el toast y marca el botón X en rojo

Método	Función
	cuando el mouse está encima.
`StartFade(tlExpired)`	Inicia el desvanecimiento. `tlExpired = .T.` significa que fue por agotarse el tiempo.
`DoOpen()`	Se activa con un clic en cualquier parte: marca el clic y luego desvanece.
`DoClose()`	Se activa con el clic en el botón X: marca el cierre y luego desvanece.
`Step(tnDt, tnMx, tnMy)`	Un paso de animación. Devuelve `.T.` cuando el toast terminó.

Nota técnica sobre el fade. VFP no admite transparencia. El efecto de desvanecimiento se logra mezclando el color del toast con el `BackColor` del host (por eso `Setup` recibe `tnHostBack`). El mejor resultado se obtiene cuando el área del toast en el formulario host tiene un fondo liso.

5.4 ucToastInbox

Bandeja de notificaciones basada en una tabla DBF. Cualquier módulo (incluso otro programa) solo necesita escribir una fila en la tabla, y el formulario host la mostrará como toast.



Creación: `CREATEOBJECT('ucToastInbox', tcDbf, tcUser)`. `tcDbf` = ruta del archivo DBF, `tcUser` = nombre del destinatario (se guarda en mayúsculas, ancho 20). La tabla se crea automáticamente si no existe y luego se abre con el alias `inbox` (shared).

Estructura de la tabla

Campo	Tipo	Contenido
`id`	C(20)	ID único automático
`tujuan`	C(20)	Destinatario (se compara con `cUser`)
`judul`	C(60)	Título del toast
`pesan`	C(250)	Texto del toast
`tipe`	N(1)	Tipo de color del 1 al 4
`gambar`	C(200)	Ruta de la imagen de avatar (opcional)
`modul`	C(20)	Nombre del módulo de origen (para navegar al hacer clic)
`refid`	C(20)	Número de referencia del documento
`dibuat`	T	Fecha y hora de creación
`dibaca`	L	Si ya fue leído o no
`dibaca_pd`	T	Fecha y hora en que se marcó como leído

Métodos

Método	Función
`Add(tcTujuan, tcJudul, tcPesan, tnType, tcGambar, tcModul, tcRefId)`	Agrega un mensaje. Devuelve el ID, o una cadena vacía si falla (ver `cLastError`).
`Fetch(toToast)`	Toma todos los mensajes no leídos de `cUser` y llama a `Notify` para los que

Método	Función
	aún no fueron tratados en esta sesión. La clave del toast = ID del mensaje.
`IsDone(tcId)`	Indica si el ID ya fue tratado (existe en `oDone`).
`MarkRead(tcId)`	Marca como leído en la tabla y en `oDone`. Devuelve `.T.` si la tabla se actualizó correctamente.
`MarkAllRead()`	Marca como leídos todos los mensajes de `cUser`.
`CountUnread()`	Cantidad de mensajes no leídos de `cUser`.
`GetInfo(tcId)`	Devuelve un objeto con `cModul` y `cRefId` para la navegación.

Propiedades: `cDbf`, `cUser`, `cLastError` (último mensaje de error), `oDone` (Collection de IDs ya tratados, para que no aparezcan duplicados aunque falle la actualización de la tabla).

Cómo otros módulos envían notificaciones

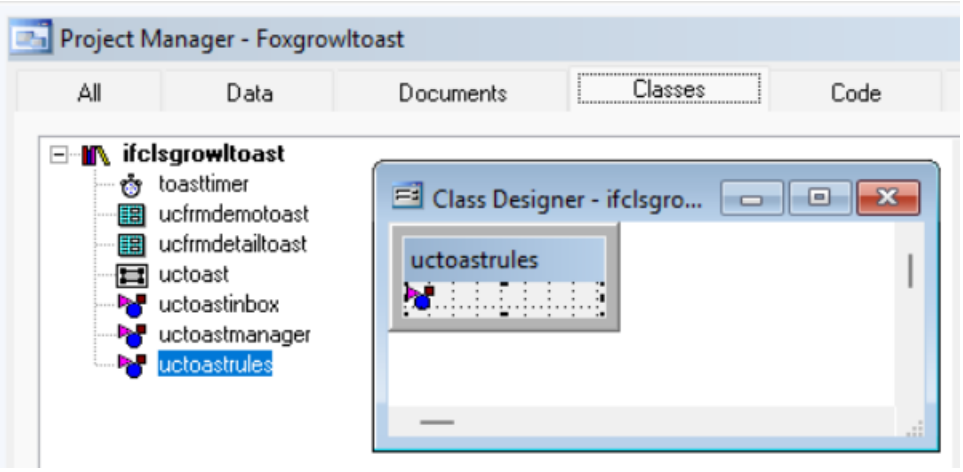
```
* Forma 1: mediante el objeto inbox
loInbox.Add('ADMIN', 'Almacén Principal', 'Hay 3 artículos
por debajo del stock mínimo.', 3, '', 'GUDANG', 'REF-001')

* Forma 2: directamente desde otro programa (sin objeto)
USE toast_inbox IN 0 SHARED AGAIN ALIAS inbox
INSERT INTO inbox (id, tujuan, judul, pesan, tipe, modul,
refid, dibuat, dibaca) ;
    VALUES (PADR(SYS(2015), 20), PADR('ADMIN', 20),
'Título', 'Texto del mensaje', 1, 'PO', 'PO-0187',
DATETIME(), .F.)
```

Requisito: el valor de `tujuan` debe ser exactamente igual (en mayúsculas) al usuario con el que se creó `ucToastInbox`.

5.5 ucToastRules

Reglas de alerta automáticas que leen dos tablas de negocio y luego muestran, actualizan o eliminan toasts según el estado más reciente de los datos.



Creación : `CREATEOBJECT('ucToastRules', tcBrg, tcPiu)` con la ruta de las tablas de artículos y de cuentas por cobrar. Si los archivos no existen, las tablas se crean **y se llenan con datos de ejemplo** (`Seed`). En producción, apunte a las tablas reales o modifique las consultas.

Estructura de las tablas leídas

- `brg` (artículos): `kode C(20)`, `nama C(60)`, `stok N(10,0)`, `satuan C(10)`
- `piu` (cuentas por cobrar): `faktur C(20)`, `customer C(60)`, `tgl D`, `jtempo D`, `sisa N(15,2)`, `status C(1)` (C = closed/pagada)

Criterios

Clave del toast	Tipo	Condición
`STK_HABIS`	4 rojo	`stok <= 0`
`STK_MIN`	3 naranja	`stok > 0` y `stok <= nMinStok`
`PIUTANG_JT`	2 azul	`status <> 'C'`, `sisa > 0`, `jtempo <= DATE()`

Propiedades

Propiedad	Valor predeterminado	Descripción
`nMinStok`	5	Límite de stock mínimo.
`nSnoozeMenit`	1	Tiempo que se pospone la alerta tras un clic o cierre (minutos).
`nHabis`, `nMinim`, `nPiutang`, `nPiutangTotal`	0	Últimos resultados calculados.
`cGlyphHabis`, `cGlyphMin`, `cGlyphPiutang`	r, i, i	Carácter Webdings del ícono de cada alerta.
`cBrg`, `cPiu`, `cLastError`		Ruta de las tablas y último mensaje de error.
`tSnzHabis`, `tSnzMin`, `tSnzPiu`	.F.	Límite de tiempo de posposición (snooze) de cada clave (DateTime).

Métodos

Método	Función
`CekStok(toToast)`	Calcula stock agotado y mínimo, luego llama a `Sync` para `STK_HABIS` y `STK_MIN`.
`CekPiutang(toToast)`	Calcula las cuentas por cobrar vencidas y su total, luego llama a `Sync` para `PIUTANG_JT`.
`Sync(toToast, tcKey, tnCount, tcTitle, tcMsg, tnType, tcGlyph)`	Núcleo de la lógica de sincronización (ver más abajo).

Método	Función
`Snooze(tcKey)` / `SnoozeAll()`	Pospone la alerta durante `nSnoozeMenit` minutos.
`ClearSnooze(tcKey)`	Cancela la posposición.
`GetSnooze` / `SetSnooze`	Lee/escribe el límite de snooze por clave.
`SqlStokKritis()` / `SqlPiutangJT()`	Devuelven el texto SQL para el formulario de detalle (cursores `crsStok` y `crsPiu`).
`Rp(tnVal)`	Formatea números con punto como separador de miles, por ejemplo 12500000 pasa a `12.500.000`.
`Seed()`, `DemoSesuaikanStok()`, `DemoLunasiTagihan()`, `DemoReset()`	Solo para demostración y datos de ejemplo.

Lógica de `Sync` (se llama en cada verificación, por defecto cada 5 segundos en la demo)

1. Cantidad de casos = 0: se limpia el snooze y se elimina el toast con esa clave (`Dismiss`). Por eso el toast desaparece solo después de ajustar el stock o saldar la factura.
2. Todavía dentro del periodo de snooze: no hace nada.
3. El toast ya existe: solo se actualiza su texto (`UpdateText`), así la cifra del toast siempre está al día sin aparecer duplicado.
4. No existe: se crea un toast nuevo mediante `Notify`.

Agregar una regla nueva (ejemplo: facturas sin pagar con más de 90 días):

```
PROCEDURE CekMacet
    LPARAMETERS toToast
    LOCAL ARRAY laX[1]
    SELECT COUNT(*) FROM piu WHERE status <> 'C' AND jtempo
< DATE() - 90 INTO ARRAY laX
```

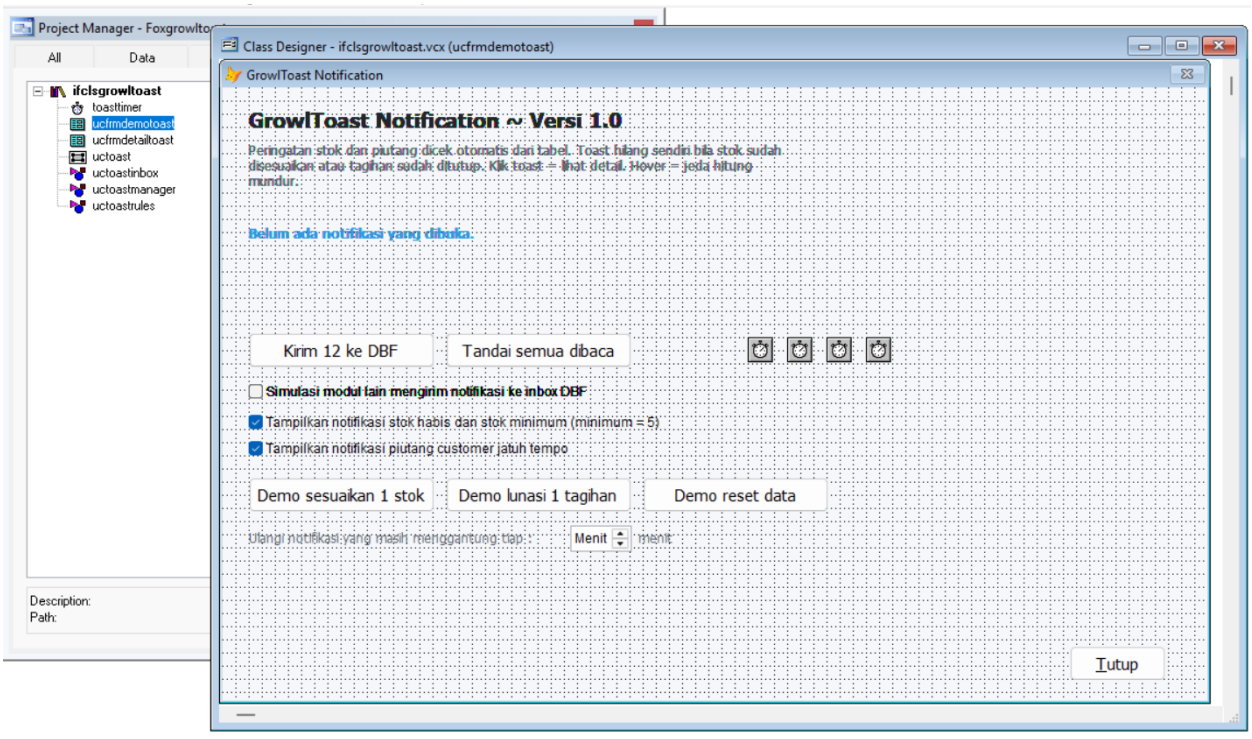
```
    This.Sync(toToast, 'PIUTANG_MACET', laX[1], 'Cuentas
Morosas', ;
        TRANSFORM(laX[1]) + ' facturas con más de 90 días de
atraso.', 4, 'r')
ENDPROC
```

Nota: `GetSnooze`/`SetSnooze` actualmente solo reconocen las tres claves predeterminadas. Una clave nueva usará el contenedor de snooze de cuentas por cobrar (`tSnzPiu`). Para un snooze independiente, agregue una propiedad y su rama correspondiente.

5.6 ucFrmDemoToast

Formulario de demostración (900x560) y a la vez ejemplo completo de cómo ser host. Se ejecuta con `DO FORM` de esta clase o con `CREATEOBJECT('ucFrmDemoToast')` seguido de `Show()` (la clase llama a `CLEAR EVENTS` al cerrarse, ver la nota en la sección 9).

Propiedades: `oToast` (manager), `oInbox`, `oRules`, `oDetail` (formulario de detalle abierto actualmente), `nDemoNo` (número de secuencia de los mensajes demo).



Timers del formulario

Timer	Intervalo	Tarea
`tmrPoll`	2000 ms	`PollInbox()`: toma mensajes nuevos de la DBF y actualiza la etiqueta de estado
`tmrEvt`	100 ms	`DispatchEvents()` y `Watchdog()` del manager
`tmrRules`	5000 ms	`CekKondisi()`: verifica stock y cuentas por cobrar
`tmrDemo`	aleatorio de 2,5 a 7,5 segundos	Envía un mensaje demo si la casilla de simulación está marcada

Métodos

Método	Función
`Load`	Configura los SET, crea `gcToastErr`, carga la librería de clases.
`Init`	Crea Inbox (usuario `ADMIN`), Rules (`nMinStok = 5`), Manager (`nMaxVisible = 5`, `nDefaultMs = 10000`), y luego hace el primer poll y la primera verificación.
`PollInbox`	`oInbox.Fetch(oToast)` y luego actualiza las etiquetas de cantidad de no leídos, visibles y en cola.
`CekKondisi`	Ejecuta `CekStok`/`CekPiutang` según las casillas. Si una casilla se desmarca, se eliminan los toasts relacionados y se limpia el snooze.
`OnToastClick`	Clave de stock/cuentas por cobrar: snooze y apertura del formulario de detalle. Clave de inbox: obtiene `modul` y `refid` y marca como leído. Incluye

Método	Función
	un ejemplo de `DO CASE` (comentado) para abrir el formulario del módulo correspondiente.
`OnToastClose`	Clave de stock/cuentas por cobrar: snooze. Clave de inbox: marca como leído.
`BukaDetail(tcFocus)`	Abre el formulario de detalle, o activa el que ya está abierto (solo una instancia).
`KirimDemo`	Agrega un mensaje aleatorio a la DBF (usuario, mensaje, módulo y tipo elegidos al azar).
`TerapkanJeda`	Copia el valor del spinner de minutos a `oToast.nRequeueMs` y `oRules.nSnoozeMenit`.
`Error`	Manejador de errores: escribe en `toast_error.log`, muestra un MessageBox e ignora errores repetidos idénticos.
`Destroy`	Apaga todos los timers, cierra el formulario de detalle, `oToast.Shutdown()`, libera las referencias.

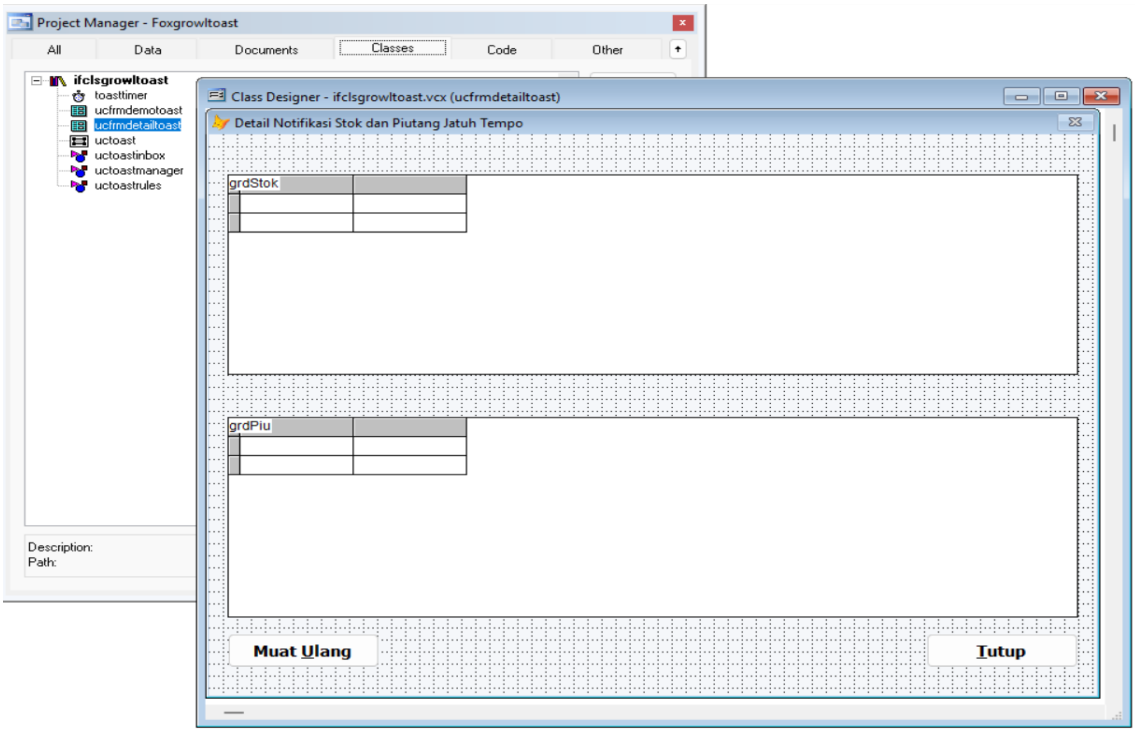
5.7 ucFrmDetailToast

Formulario de detalle de 760x560 con dos grids: stock agotado/mínimo (arriba) y cuentas por cobrar vencidas (abajo). Usa `DataSession = 2` (sesión de datos privada), por lo que en `Init` abre por sí mismo las tablas `brg` y `piu` (shared).

- `Init(toOwner, toRules, tcFocus)`: guarda las referencias, abre las tablas y luego llama a `Aktifkan(tcFocus)`.
- `Aktifkan(tcFocus)`: colorea el título de la sección según el toast en el que se hizo clic (rojo para `STK_HABIS`, naranja para `STK_MIN`, azul para `PIUTANG_JT`) y luego llama a `LoadData()`.
- `LoadData`: ejecuta el SQL de `oRules.SqlStokKritis()` y `SqlPiutangJT()` en los cursores `crsStok` y `crsPiu`, y completa los

títulos con la cantidad de artículos, la cantidad de facturas y el total en rupias.

- `FormatGrids`: ancho y título de columnas, y coloreado de filas: HABIS (agotado) = rosa claro; atraso de 30 días o más = rosa claro; atraso de 1 a 29 días = naranja claro.
- El botón **Muat Ulang** (Recargar) llama a `LoadData`, **Tutup** (Cerrar) libera el formulario. `Destroy` vacía `oOwner.oDetail`.



6. Cómo Leer la Pantalla de Demostración

Elemento en pantalla	Explicación
Título y descripción	Etiquetas estáticas. Explican el comportamiento: desaparece solo cuando los datos se corrigen, clic = detalle, hover = pausa.
Texto azul `Belum ada notifikasi yang dibuka.` (Aún no se ha abierto ninguna	`lblLog`: rastro de la última acción (toast abierto, resultado de la demo, etc.).

Elemento en pantalla	Explicación
notificación)	
`Inbox belum dibaca / Tampil / Antri` (Inbox sin leer / Visibles / En cola)	`lblStatus`: cantidad de mensajes DBF sin leer, toasts activos en pantalla y toasts en cola.
`Stok habis / Stok minimum / Piutang JT` (Stock agotado / Stock mínimo / Cuentas por cobrar vencidas)	`lblKondisi`: resultado del cálculo de las reglas. Con los datos de ejemplo: stock agotado 2 (BRG-001, BRG-002), mínimo 4 (BRG-003 a BRG-006), 5 facturas vencidas.
Botón `Kirim 12 ke DBF` (Enviar 12 a DBF)	Escribe 12 mensajes aleatorios en la tabla inbox.
Botón `Tandai semua dibaca` (Marcar todo como leído)	Marca todo el inbox como leído, pospone todas las alertas de las reglas y limpia todos los toasts.
Casilla de simulación	Si está marcada, `tmrDemo` envía mensajes aleatorios automáticamente como si vinieran de otro módulo.
Dos casillas de stock y cuentas por cobrar	Activan o desactivan cada regla.
`Demo sesuaikan 1 stok` (Demo ajustar 1 stock)	Sube a 100 el stock de un artículo crítico; el toast de stock se ajusta o desaparece.
`Demo lunasi 1 tagihan` (Demo saldar 1 factura)	Cierra una factura vencida (status C, sisa 0).
`Demo reset data`	Devuelve los datos de stock y cuentas por cobrar a su estado inicial.
Spinner de minutos	Intervalo de repetición de las notificaciones pendientes (`nRequeueMs` y `nSnoozeMenit`).

Elemento en pantalla	Explicación
Línea gris de diagnóstico	`DiagText()`: estado del timer de toasts, próxima aparición e intervalo de repetición.
Tarjetas toast a la derecha	Máximo 5 visibles (según `nMaxVisible`); el resto cuenta en `Antri` (en cola).

7. Ciclo de Vida de un Toast

1. **Creación**: `Notify` mete el elemento en `oQueue` con `nWaitUntil = 0`.
2. **Aparición**: `Fill` encuentra un espacio libre, `Spawn` crea el `ucToast` y el toast sube hasta su posición.
3. **Cuenta regresiva**: la barra se reduce según `nMs`. Mouse sobre el toast = el conteo se detiene.
4. **Fin**, por una de estas causas:
    - *Tiempo agotado*: estado 4, luego se desvanece por turnos y después `Retire(expired)`. El elemento vuelve a la cola y reaparece tras `nRequeueMs` (mientras no supere `nMaxRepeat`).
    - *Clic*: fade y se envía `OnToastClick` al host. No se repite.
    - *Cierre (X)*: fade y se envía `OnToastClose` al host. No se repite.
    - *Dismiss/ClearAll/condición resuelta*: fade sin evento.
5. **Repetición en las reglas**: tras un clic o cierre, la alerta entra en snooze. Cuando termina el snooze y la condición persiste, `CekKondisi` la vuelve a mostrar.

8. Lista de Verificación para Integrar en su Propia Aplicación

1. Copie `ifclsgrowltoast.vcx` (y `.vct`) a la carpeta del proyecto, agréguelo al proyecto y ejecute `SET CLASSLIB TO ... ADDITIVE`.
2. En el formulario host: agregue las propiedades `oToast`, `oInbox` (si se necesita), `oRules` (si se necesita), `oDetail` (si se necesita).
3. Cree el manager en `Init` y configure `nMaxVisible`, `nDefaultMs`, `nRequeueMs`.

4. Agregue un timer de 100 ms que llame a `DispatchEvents()` y `Watchdog()`.
5. Agregue los métodos `OnToastClick` y `OnToastClose`.
6. Si usa inbox: cree `ucToastInbox` con el usuario correspondiente y llame a `Fetch(oToast)` desde un timer periódico (por ejemplo, cada 2 segundos).
7. Si usa reglas: cambie las consultas/tablas de `ucToastRules` según su base de datos real y llame a `CekStok`/`CekPiutang` desde un timer (por ejemplo, cada 5 segundos).
8. Llame a `oToast.Shutdown()` en el `Destroy`.

9. Notas Importantes y Trampas

1. **`CLEAR EVENTS` en `ucFrmDemoToast.Destroy`.** Adecuado para una demo independiente, pero si este formulario o su código se usa dentro de una aplicación que utiliza `READ EVENTS`, cerrar el formulario detendrá toda la aplicación. Elimine esa línea al integrarlo.
2. **Los toasts viven solo dentro del formulario host.** No flotan sobre otras aplicaciones. Asegúrese de que el formulario sea lo bastante alto; la capacidad se calcula a partir de la altura del host.
3. **`DispatchEvents` y `Watchdog` deben llamarse obligatoriamente** desde el timer del host. Sin ellos, el clic y el cierre nunca llegan al host.
4. **`Shutdown()` es obligatorio** en el `Destroy` del host para liberar limpiamente el timer y las referencias.
5. **`nToastW` y `nToastH`** del manager deben coincidir con el tamaño de `ucToast` (340x84) si se cambia el tamaño de la tarjeta.
6. **Datos de ejemplo.** `Seed` solo se ejecuta cuando el archivo de tabla aún no existe. No apunte a una base de datos de producción sin comprender esto.
7. **Sesión de datos.** `ucToastInbox` y `ucToastRules` tienen una propiedad llamada `datasession`; en estas clases Custom probablemente sea solo una propiedad común (no una sesión de datos privada), por lo que el objeto se ejecuta en la sesión de datos donde fue creado. Los alias `inbox`, `brg`, `piu` se usan en esa sesión; evite conflictos con otros alias. `ucFrmDetailToast` usa `DataSession = 2`, por lo que abre sus propias tablas.
8. **La variable `PUBLIC gcToastErr`** solo la usa el manejador de errores de la demo. Se elimina en `Destroy`.

9. **La fuente Webdings** debe estar instalada. Si los íconos aparecen como cuadros o letras normales, la fuente no está instalada.
10. **Las tablas se abren en modo shared** (`SET EXCLUSIVE OFF`). Si otro proceso mantiene la tabla en modo exclusivo, aparece el error `File is in use`.

10. Solución de Problemas

Síntoma	Posible causa y solución
El toast no aparece	La librería de clases no se cargó con `SET CLASSLIB`; no se llamó a `Notify`; la misma clave sigue activa o en cola (`Exists`); revise con `DiagText()` si el timer de toasts está activo.
El toast aparece pero el clic no reacciona	El timer de 100 ms no llama a `DispatchEvents`, o el método `OnToastClick` no existe en el host.
El toast aparece una y otra vez	Es el comportamiento de requeue. Haga clic o cierre el toast, o configure `nMaxRepeat` y `nRequeueMs`. En las reglas, ajuste los minutos de snooze.
El toast se queda detenido y no se mueve	Timer bloqueado. Verifique que `Watchdog()` se llame desde el timer del host.
El toast aparece cortado o solo en parte	El formulario host es demasiado bajo. Aumente la altura del formulario o reduzca `nMaxVisible`.
Los mensajes del inbox nunca aparecen	El valor de `tujuan` no coincide con el usuario de `ucToastInbox`, o `dibaca` ya es `.T.`. Revise `cLastError`.
Error `File is in use`	La tabla está bloqueada en modo

Síntoma	Posible causa y solución
	exclusivo por otro proceso.
Error registrado en archivo	Revise `toast_error.log` en la carpeta de la aplicación (lo escribe `ucFrmDemoToast.Error`).